

Recap:

How to print "Hello, World"

```
#include <stdio.h>
```

```
int main() {
```

```
    printf("Hello, World");
```

```
    return 0;
```

```
}
```

→ C is case sensitive ('a' is different from 'A')

printf is a function to output (into the terminal)

scanf is a function to take input

(from the user)

scanf stores the input in a defined space.

in C, to store some information, we need a

'variable'. A variable needs to be declared before we use it, so that space can be allocated for it.

Ex. 'int a;' declares an integer 'a'.

Ex. programme with printf and scanf

```
#include <stdio.h>
```

```
int main() {
```

```
    int a = 5; // Declare + Define
```

```
    printf("value of a is %d\n", a); // prints 5
```

```
    int b; // only declaration.
```

```
    scanf("%d", &b);
```

```
    printf("value of b is %d\n", b);
```

return 0;

}

what does the syntax of scanf mean here?

```
scanf("%d", &b);
```

here, the first argument is the string format, i.e., what type of input should it look for?

"%d" means look for one integer in the input
the second argument is a memory address, i.e., where should it store the input?

&b means it should store it in the space allocated for b.

in C, you need to explicitly "declare" any variable. This basically means you are asking for some memory (space) to store something. After you declare it, you use the name to refer to the value in the space.

To refer to the space itself, we use &. So &b means the address of b, or the space in which it is stored.

Another example: Take two numbers, add and print

```
int a, b; // declares 2 integers. int a; int b; is also correct
scanf("%d%d", &a, &b); // takes 2 integer inputs
printf("%d", a+b);
```


Variable naming convention in C:
variables can contain

- a to z
- A to Z
- 0 to 9
- _ (underscore)

variables cannot contain

- spaces, newlines, tabs
- special characters
- punctuation

variables cannot be keywords.

variables cannot start with digits

Ex. of valid and invalid names

v1, v2

1v, 2v

a-number

a number, a-number

atnum

@num

integer, forks, four

int, for, char

INTEGER

INT. EG ER

In C, there are also several operators

Unary Operators

a++; // or ++a; → increases a by one

a--; // or --a; → decreases a by one

(value then operator vs. operator then value)

Binary Operators

Arithmetic → +, -, *, /, %

add, subtract, multiply, divide, modulus

Comparison $<, <=, >, >=, ==, !=$
Logical (conditions) $&\&, ||, !$ (and, or, not)
Assignment $=, +=, -=, /=, *=, \%=$
Bitwise $\&, |, \sim, \wedge, <<, >>$
(AND, OR, NOT, XOR, left shift, right shift)

Other than assignment operators, the other binary operators don't actually change the value of the variable.

Ex. `int a = 5;`
`a++;` // now `a = 6`
`a = a + 5;` // `a = 11`
`a -= 2;` // `a = a - 2 = 11 - 2 = 9`
`a = a & a;` // `a & a = a` (bitwise)

Booleans

Booleans can take value 0 or 1.

What is true? $\rightarrow$ 1, any non-zero integer,
any non-zero character,
any non-zero float

What is false? $\rightarrow$ 0, 0.0, +0.0, -0.0, "\0"

Logical (condition) operators take two boolean inputs,
and output ^aboolean value
comparison operators take as input two integers and
outputs a bool.

	1 byte = 8 bits		
%d	int → 4B	Range -2^{31} to $+2^{31}-1$	unsigned (0 to $2^{32}-1$)
%c	char → 1B	-2^7 to 2^7-1	(0 to 2^8-1)
	bool → 1B	"	"
%hd	short (int) → 2B	-2^{15} to $2^{15}-1$	(0 to $2^{16}-1$)
%ld	long (int) → 8B	:	:
%f	float 4B	:	:
%lf	double 8B	:	:
%u	(unsigned - range is positive)		

Characters - Governed by ASCII

ASCII maps values from 0 to 2^8-1 to specific characters
(Look up ASCII table online for exact correspondence)

%zu, sizeof()

→ tells you the size of the data type of a variable

%b → prints binary value

%x → prints hexadecimal value

Floating Point Representation

→ separate topic

floats are decimals.

1 bit sign 8 bits exponent 23 bits mantissa